გაკვეთილი 6 Mod 7 (7.2)

გაკვეთილი 2 — Git Branch-ის შექმნა, გამოყენება და მართვა

2.1. რა არის Git Branch?

წარმოიდგინეთ, რომ სამი მოსწავლე ერთად ქმნის სკოლის ვებსაიტს.

პროექტში არსებობს შემდეგი ფაილები:

```
school-website/
    index.html
    style.css
    script.js
```

სამ მოსწავლეს სხვადასხვა დავალება აქვს:

- პირველი მოსწავლე ქმნის ვებსაიტის Header-ს;
- მეორე მოსწავლე მუშაობს Footer-ზე;
- მესამე მოსწავლე ქმნის Contact გვერდს.

თუ ყველა მოსწავლე ერთსა და იმავე ძირითად ვერსიაზე იმუშავებს, შესაძლოა ერთმანეთის ცვლილებები შეუთავსებელი აღმოჩნდეს ან შემთხვევით შეიცვალოს სხვისი კოდი.

ამ პრობლემის მოსაგვარებლად Git-ში გამოიყენება Branch (განშტოება).

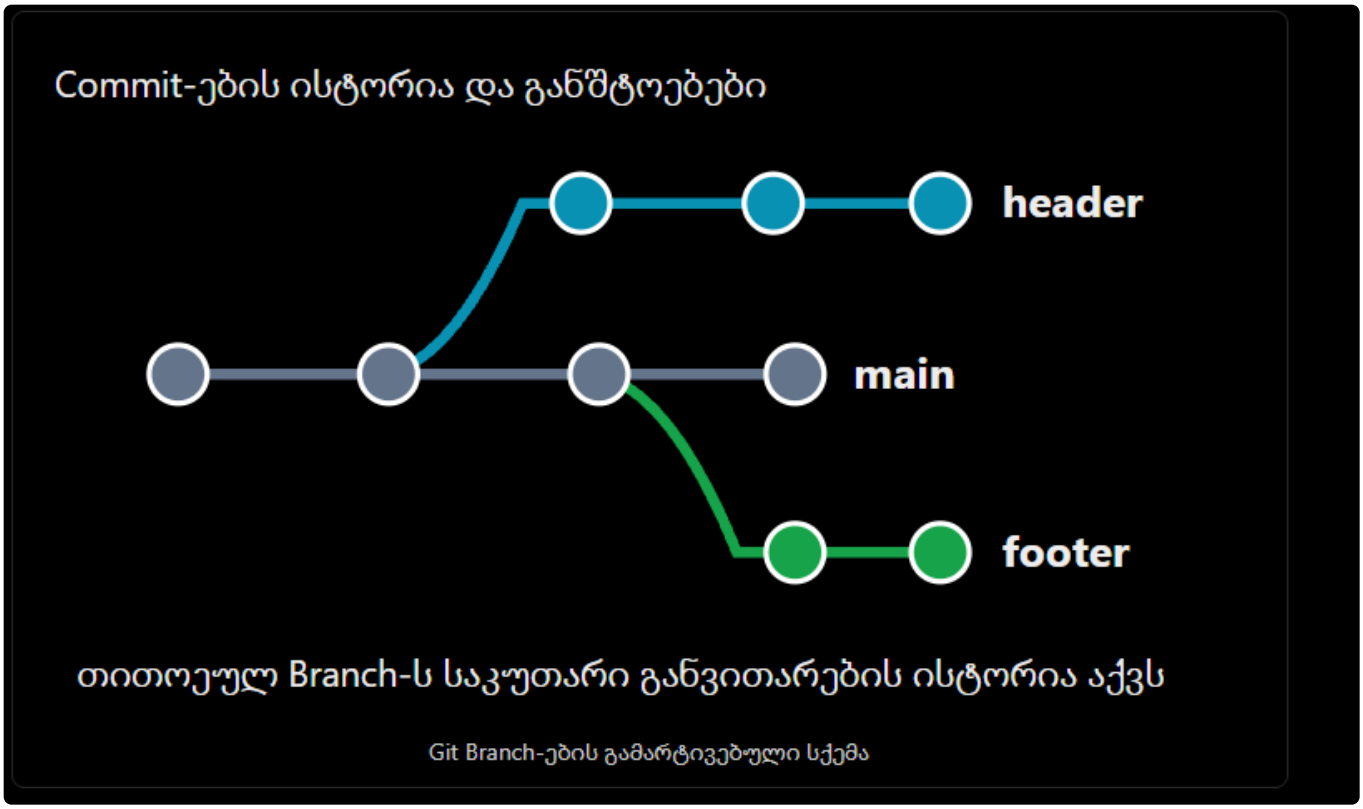
Branch საშუალებას გვაძლევს, პროექტის განვითარება სხვადასხვა მიმართულებით გავაგრძელოთ.

მაგალითად:

- `main` — პროექტის ძირითადი ვერსია;
- `header` — განშტოება Header-ის შესაქმნელად;
- `footer` — განშტოება Footer-ის შესაქმნელად;
- `contact` — განშტოება Contact გვერდისთვის.

Branch-ის შექმნისას პროექტის ისტორიისგან დამოუკიდებელ სამუშაო მიმართულებას ვიღებთ, რომელშიც შეგვიძლია ახალი commit-ები შევქმნათ.

როგორ გამოიყურება Branch-ები?



აქ ყველა განშტოება ერთი პროექტიდან იწყება, მაგრამ შემდგომ დამოუკიდებლად ვითარდება.

მნიშვნელოვანია: Branch არ არის პროექტის ცალკე ფიზიკური ასლი. ის Git-ის ისტორიაში კონკრეტულ commit-ზე მიმთითებელია, რომელიც ახალი commit-ების შექმნისას წინ გადაადგილდება.

2.2. რა არის Main Branch?

`main` ჩვეულებრივ პროექტის მთავარი Branch-ია.

მასში ინახება პროექტის ძირითადი ვერსია.

მაგალითად:

```
main
  ├── index.html
  ├── style.css
  └── script.js
```

თუ გვინდა ახალი ფუნქციონალის დამატება, კარგი პრაქტიკაა, ჯერ ახალი Branch შევქმნათ და ცვლილებები იქ განვახორციელოთ.

მაგალითად, ვებგვერდზე ნავიგაციის მენიუს დამატება:

```
main
  |
  +---- navbar
```

`navbar` Branch-ში შეგვიძლია HTML და CSS კოდი შევცვალოთ ისე, რომ `main` Branch-ის ისტორია უცვლელი დარჩეს.

როდესაც მუშაობას დავასრულებთ, შემდეგ გაკვეთილში ვისწავლით ცვლილებების მთავარ Branch-ში გაერთიანებას — `git merge`.

2.3. არსებული Branch-ების ნახვა — git branch

პირველ რიგში უნდა ვიცოდეთ, რომელ Branch-ში ვმუშაობთ.

ამისთვის ვიყენებთ:

```
git branch
```

მაგალითად, შედეგი შეიძლება იყოს:

```
* main
  header
  footer
```

აქ ვარსკვლავი `*` გვიჩვენებს, რომელ Branch-ში ვიმყოფებით.

ამ შემთხვევაში აქტიურია `main`.

თუ გადავერთვებით `header` Branch-ზე, მივიღებთ:

```
  main
* header
  footer
```

ბრძანება `git branch` აჩვენებს ლოკალურ Branch-ებს.

დისტანციური Branch-ების სანახავად შეგვიძლია გამოვიყენოთ:

```
git branch -r
```

ხოლო ყველა ცნობილი ლოკალური და დისტანციური Branch-ის სანახავად:

```
git branch -a
```

2.4. ახალი Branch-ის შექმნა

ახალი Branch-ის შესაქმნელად ვიყენებთ:

```
git branch branch-name
```

მაგალითად, გვინდა Header-ის შესაქმნელად ახალი Branch.

```
git branch header
```

ამით შეიქმნება ახალი Branch სახელით `header`.

თუ ახლა გავუშვებთ:

```
git branch
```

მივიღებთ:

```
  header
* main
```

აქ მნიშვნელოვანია ერთი დეტალი:

`git branch header` ქმნის ახალ Branch-ს, მაგრამ ავტომატურად მასზე არ გადაგვიყვანს.

ამისთვის გვჭირდება `git checkout`.

2.5. ერთი Branch-იდან მეორეზე გადასვლა — git checkout

`git checkout` ბრძანების საშუალებით შეგვიძლია გადავიდეთ სხვა Branch-ზე.

მაგალითად:

```
git checkout header
```

ამის შემდეგ აქტიური Branch გახდება `header`.

ტერმინალში შეიძლება გამოჩნდეს:

```
Switched to branch 'header'
```

ახლა თუ შევამოწმებთ:

```
git branch
```

შედეგი იქნება:

```
* header
  main
```

ეს ნიშნავს, რომ ამჟამად ვმუშაობთ `header` Branch-ში.

თუ ისევ მთავარ Branch-ზე დაბრუნება გვინდა:

```
git checkout main
```

გასათვალისწინებელია: თუ სამუშაო ფაილებში შეუნახავი ცვლილებები გვაქვს, რომლებზეც Branch-ის შეცვლა იმოქმედებს, Git-მა შეიძლება გადასვლა არ დაუშვას. ასეთ შემთხვევაში ჯერ ცვლილებები უნდა შევინახოთ commit-ში ან სხვა უსაფრთხო გზით.

2.6. Branch-ის შექმნა და მასზე გადასვლა ერთი ბრძანებით

წინა მაგალითში ახალი Branch-ის შესაქმნელად ორი ბრძანება გამოვიყენეთ:

```
git branch header
git checkout header
```

მაგრამ Git გვაძლევს საშუალებას, ორივე მოქმედება ერთი ბრძანებით შევასრულოთ:

```
git checkout -b header
```

ეს ბრძანება ერთდროულად:

1. ქმნის Branch-ს სახელით `header`;
2. ავტომატურად გადადის ახალ Branch-ზე.

მაგალითად, გვინდა Footer-ის შექმნა:

```
git checkout main
```

```
git checkout -b footer
```

ახლა შედეგი იქნება:

```
  header
  main
* footer
```

ეს მეთოდი განსაკუთრებით მოსახერხებელია გუნდურ პროექტებში.

შენიშვნა: Git-ის თანამედროვე ვერსიებში Branch-ებს შორის გადასასვლელად არსებობს `git switch`, ხოლო ახალი Branch-ის შესაქმნელად — `git switch -c`. ამ გაკვეთილში ყურადღებას ვამახვილებთ `git checkout`-ზე, რადგან ის სასწავლო მასალაშია მოცემული და ფართოდ გამოიყენება.

2.7. ცვლილებების შეტანა საკუთარ Branch-ში

ახლა წარმოვიდგინოთ, რომ ვართ `header` Branch-ში.

ვამოწმებთ:

```
git branch
```

შედეგი:

```
* header
  main
```

ვხსნით `index.html` ფაილს და ვამატებთ Header-ს.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>School Website</title>
</head>
<body>

    <header>
        <h1>Khoni School</h1>
        <nav>
            <a href="index.html">Home</a>
            <a href="about.html">About</a>
```

```
            <a href="contact.html">Contact</a>
        </nav>
    </header>

</body>
</html>
```

ამის შემდეგ ვამოწმებთ:

```
git status
```

Git გვაჩვენებს, რომ შეიცვალა `index.html`.

მაგალითად:

```
modified: index.html
```

ახლა ცვლილებებს ვინახავთ Git-ის ისტორიაში.

```
git add index.html
git commit -m "Added website header"
```

ეს commit შეიქმნება `header` Branch-ში.

თუ ამ დროს გადავალთ `main` Branch-ზე:

```
git checkout main
```

`header` Branch-ში შესრულებული commit არ იქნება `main`-ის ისტორიაში და პროექტი გამოჩნდება მისი ძველი მდგომარეობით.

თუ ისევ დავბრუნდებით:

```
git checkout header
```

ჩვენი Header ისევ გამოჩნდება.

ეს არის Branch-ის მთავარი უპირატესობა: შეგვიძლია სხვადასხვა ფუნქციონალზე ვიმუშაოთ და ცვლილებები ერთმანეთისგან განცალკევებულად შევინახოთ.

2.8. Branch-ის სახელის შეცვლა და წაშლა

Branch-ების მართვისას ზოგჯერ საჭიროა მათი სახელების შეცვლა ან არასაჭირო Branch-ის წაშლა.

Branch-ის სახელის შეცვლა

თუ ამჟამად ვართ `header` Branch-ში და გვსურს მას `navbar` დავარქვათ:

```
git branch -m navbar
```

ამის შემდეგ Branch-ის ახალი სახელი იქნება `navbar`.

Branch-ის წაშლა

თუ გვინდა არასაჭირო ლოკალური Branch-ის წაშლა:

```
git branch -d navbar
```

მაგრამ ჯერ სხვა Branch-ში უნდა გადავიდეთ:

```
git checkout main
git branch -d navbar
```

`-d` უსაფრთხო წაშლის ვარიანტია: Git არ წაშლის Branch-ს, თუ მისი ცვლილებები ჯერ სათანადოდ არ არის გაერთიანებული.

არსებობს იძულებითი წაშლაც:

```
git branch -D navbar
```

ეს ბრძანება სიფრთხილით უნდა გამოვიყენოთ, რადგან შეიძლება დაგვეკარგოს ისეთი commit-ები, რომლებიც სხვა Branch-ში არ არის შენარჩუნებული.

სასწავლო პროექტებში უმჯობესია, ძირითადად `-d` გამოვიყენოთ.

3. სრული პრაქტიკული მაგალითი — სკოლის ვებსაიტზე მუშაობა

ამ მაგალითში შევქმნით ორ Branch-ს: `header` და `footer`.

ნაბიჯი 1 — არსებული პროექტის გახსნა

ვხსნით პირველი გაკვეთილიდან დაკლონილ პროექტს.

VS Code-ის Terminal-ში:

```
git status
git branch
```

ვრწმუნდებით, რომ ვართ `main` Branch-ში და სამუშაო ფაილებში შეუნახავი ცვლილებები არ გვაქვს.

ნაბიჯი 2 — Header Branch-ის შექმნა

```
git checkout -b header
```

ახლა ვხსნით `index.html` ფაილს და ვამატებთ:

```
<header>
    <h1>Welcome to Our School</h1>
    <nav>
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Contact</a>
    </nav>
</header>
```

ვინახავთ ცვლილებას:

```
git add index.html
git commit -m "Added header"
```

ნაბიჯი 3 — Main Branch-ზე დაბრუნება

```
git checkout main
```

გავხსნათ `index.html`.

დავინახავთ, რომ Header-ის დამატება `main` Branch-ში არ ასახულა.

ეს იმიტომ, რომ ის მხოლოდ `header` Branch-ში დავამატეთ და commit-ით შევინახეთ.

ნაბიჯი 4 — Footer Branch-ის შექმნა

```
git checkout -b footer
```

ახლა `index.html` ფაილში ვამატებთ:

```
<footer>
    <p>© 2026 Khoni School</p>
    <p>All rights reserved.</p>
</footer>
```

ცვლილების შენახვა:

```
git add index.html
git commit -m "Added footer"
```

ნაბიჯი 5 — Branch-ების შემოწმება

```
git branch
```

შედეგი:

```
* footer
  header
  main
```

ამ მომენტში გვაქვს სამი Branch:

Branch	მდგომარეობა
`main`	პროექტის საწყისი ვერსია
`header`	დამატებულია Header
`footer`	დამატებულია Footer

ახლა შეგვიძლია თითოეულ Branch-ზე გადავიდეთ და დავაკვირდეთ განსხვავებებს.

მაგალითად:

```
git checkout header
```

გამოჩნდება Header-ის ვერსია.

```
git checkout footer
```

გამოჩნდება Footer-ის ვერსია.

```
git checkout main
```

დავბრუნდებით ძირითად ვერსიაზე.

ამ ეტაპზე Branch-ებს არ ვაერთიანებთ. ეს მესამე გაკვეთილის თემაა.

4. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — ახალი Branch-ის შექმნა (მარტივი)

დავალება: შექმენით ახალი Branch სახელით `about-page`.

მოთხოვნები:

1. გახსენით პირველი გაკვეთილიდან დაკლონილი პროექტი.
2. გადადით `main` Branch-ზე.
3. შექმენით ახალი `about-page` Branch.
4. გადადით ახალ Branch-ზე.
5. შეამოწმეთ აქტიური Branch.

გამოიყენეთ:

```
git checkout main
git checkout -b about-page
git branch
```

მოსალოდნელი შედეგი:

```
* about-page
  main
```

(შესაძლოა სხვა Branch-ებიც გამოჩნდეს.)

სავარჯიშო 2 — About გვერდის დამატება (საშუალო)

დავალება: `about-page` Branch-ში შექმენით ახალი HTML გვერდი.

მოთხოვნები:

1. შექმენით `about.html` ფაილი.
2. დაამატეთ HTML-ის ძირითადი სტრუქტურა.

3. ჩაწერეთ სკოლის სახელი `h1` სათაურში.
4. დაამატეთ ორი პარაგრაფი სკოლის შესახებ.
5. დაამატეთ ბმული მთავარ გვერდზე გადასასვლელად.
6. ცვლილებები შეინახეთ commit-ში.

მაგალითი:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>About Our School</title>
</head>
<body>

    <h1>About Khoni School</h1>

    <p>Our school teaches many subjects.</p>
    <p>We are learning web development.</p>

    <a href="index.html">Back to Home</a>

</body>
</html>
```

ცვლილებების შენახვა:

```
git add about.html
git commit -m "Created about page"
```

შემდეგ გადადით `main` Branch-ზე და შეამოწმეთ, არსებობს თუ არა იქ `about.html`.

მოსალოდნელი შედეგი: `main` Branch-ში ეს ფაილი არ უნდა გამოჩნდეს, რადგან ის მხოლოდ `about-page` Branch-ში დავამატეთ და commit-ით შევინახეთ.

სავარჯიშო 3 — სამი Branch-ის მართვა (რთული)

დავალება: შექმენით სამი განსხვავებული Branch და თითოეულში დამოუკიდებელი ცვლილება შეიტანეთ.

Branch	დავალება
`navbar`	შექმენით ნავიგაციის მენიუ

Branch	დავალება
`gallery`	შექმენით `gallery.html` გვერდი
`contact`	შექმენით `contact.html` გვერდი

მოთხოვნები:

1. სამივე Branch შექმენით `main` Branch-იდან.
2. თითოეულ Branch-ში შეიტანეთ შესაბამისი ცვლილებები.
3. ყველა Branch-ში გააკეთეთ მინიმუმ ერთი commit.
4. `git branch` ბრძანებით შეამოწმეთ, რომ სამივე არსებობს.
5. გადადით Branch-ებს შორის და შეადარეთ ფაილები.
6. ბოლოს დაბრუნდით `main` Branch-ზე.

დამატებითი დავალება: გამოიყენეთ:

```
git log --oneline --all --graph --decorate
```